SERVICIO NACIONAL DE APRENDIZAJE (SENA) • REGIONAL DISTRITO CAPITAL

Informe Técnico: Herramientas de Versionamiento (GIT) Instalada y Configurada

Evidencia de Aprendizaje: GA7-220501096-AA1-EV05 • Programa ADSO

APRENDIZ:	PROGRAMA:	REPOSITORIO REMOTO:	PLATAFORMA WEB:
Gustavo Gómez	ADSO (2026)	Programa-Git	GitHub / GitLab

1. INTRODUCCIÓN

En el ciclo de vida del desarrollo de software moderno y dentro del paradigma DevOps, la Integración Continua (CI) se fundamenta en la capacidad de los equipos de ingenieros para sincronizar sus avances de forma rápida, aislada e inalterable **(SENA, 2024)**. Los Sistemas de Control de Versiones Distribuidos (DVCS), liderados por la tecnología Git, permiten desvincular la dependencia del almacenamiento estrictamente local, permitiendo que cada desarrollador conserve una réplica completa de la base de datos de objetos mientras interactúa con repositorios remotos alojados en la nube.

El presente informe documenta el procedimiento práctico y la fundamentación conceptual para la conexión del entorno local de versionamiento Git con una plataforma remota (GitHub / GitLab) mediante la ejecución rigurosa de comandos por interfaz de línea de comandos (CLI). Se incluye el desglose de comandos para la vinculación remota, la verificación de endpoints de red, el envío del árbol de confirmaciones iniciales, la redirección de flujos para la creación de archivos y la sincronización incremental de cambios.

2. JUSTIFICACIÓN

El trabajo en proyectos de desarrollo de software sin un mecanismo de sincronización remota genera serios riesgos de pérdida de datos por fallos físicos de almacenamiento local y dificulta la revisión colaborativa de código **(Atlassian, s.f.)**.

La configuración de repositorios remotos como `Programa-Git` establece un punto centralizado de auditoría, integración y respaldo. El dominio riguroso de la sintaxis CLI de Git (comandos `git remote`, `git add`, `git commit` y `git push`) asegura que el desarrollador comprenda el flujo interno entre el directorio de trabajo (Working Directory), el área de preparación (Staging Area) y la base de datos distribuida (Git Directory), garantizando la integridad de los artefactos producidos bajo los lineamientos de la industria de software.

3. OBJETIVOS

3.1 Objetivo General

Conectar el entorno local de control de versiones Git con la plataforma remota de hospedaje de código (GitHub/GitLab) mediante la ejecución paso a paso de comandos de consola, demostrando la comprensión teórica de la vinculación remota, el empaquetado de estados y la actualización incremental de ramas.

3.2 Objetivos Específicos

- Crear e inicializar un repositorio público denominado `Programa-Git` en la plataforma remota seleccionada.
- Configurar el puntero remoto (`remote add`) en la máquina local y verificar las direcciones URL de lectura y escritura mediante la bandera `-v`.
- Publicar la rama principal (`main` / `master`) y su árbol de confirmaciones en el servidor remoto mediante `git push`.
- Generar el artefacto `kotlin.txt` utilizando redirección de entradas/salidas en la consola, empaquetar sus cambios en el staging area, registrar la confirmación con metadatos y sincronizar el estado final con el repositorio remoto.

4. DESARROLLO DE LA ACTIVIDAD Y EXPLICACIÓN RIGUROSA DE COMANDOS

A continuación se presenta el procedimiento técnico ejecutado para dar cumplimiento a los tres pasos normados por el taller del SENA:

Paso 1: Creación del Repositorio Remoto y Vinculación Local

Se creó un nuevo repositorio público en la plataforma remota bajo el nombre `Programa-Git`. Posteriormente, desde la consola Git Bash en el directorio local del proyecto, se procedió a vincular y verificar la referencia remota **(Git, 2021)**.

COMANDO 1.1 `git remote add github <URL_HTTPS>`

Explicación Técnica Rigurosa: Registra un nuevo alias de servidor remoto en la sección `[remote]` del archivo de configuración interno `.git/config` del repositorio local. El parámetro `github` funciona como una etiqueta abreviada que reemplaza la URL completa en ejecuciones futuras. La URL HTTPS especifica la dirección de socket sobre la cual se resolverán las peticiones de red para la transferencia de objetos **(Git, 2021)**.

COMANDO 1.2 `git remote -v`

Explicación Técnica Rigurosa: La bandera `-v` (del inglés *verbose*) fuerza a Git a inspeccionar las referencias del sistema y retornar en la salida estándar (stdout) los nombres de los remotos configurados acompañados de sus URLs físicas. Muestra dos endpoints independientes: uno asignado a operaciones de lectura/extracción (`fetch`) y otro a operaciones de escritura/subida (`push`) **(Atlassian, s.f.)**.

Paso 2: Publicación Inicial del Repositorio Local al Remoto

Una vez establecida la ruta remota, se envió el historial de confirmaciones acumuladas en el entorno local hacia el servidor distante.

COMANDO 2.1 `git push github main`

Explicación Técnica Rigurosa: El comando `git push` empaqueta los objetos comprimidos locales (blobs, árboles y confirmaciones SHA-1) y los transfiere a través de la red hacia el servidor apuntado por el alias `github` **(Git, 2021)**. El argumento `main` indica la rama origen y destino sobre la cual se actualizará el puntero de referencia remota (`refs/remotes/github/main`). Durante esta fase se ejecuta la autenticación transparente mediante `Git Credential Manager` para validar los permisos de escritura del desarrollador.

Paso 3: Creación de kotlin.txt, Staging, Commit y Push Incremental

Se creó el archivo de datos del usuario y se realizó el ciclo completo de preparación, confirmación e integración remota.

COMANDO 3.1 `cat > kotlin.txt`

Explicación Técnica Rigurosa: Utiliza el utilitario POSIX `cat` combinado con el operador de redirección de salida `>`. Este operador desvía el flujo de entrada estándar (stdin) de la terminal y crea físicamente el archivo `kotlin.txt` en el directorio de trabajo (Working Directory). Al ingresar las líneas conteniendo el nombre y correo y cerrar el buffer (mediante `Ctrl + C` o `Ctrl + D`), el archivo queda registrado en estado *Untracked* (no rastreado).

COMANDO 3.2 `git add .`

Explicación Técnica Rigurosa: Inspecciona recursivamente el directorio de trabajo actual y calcula el hash de los archivos nuevos o modificados, creando los correspondientes objetos *blob* comprimidos dentro de la base de datos `.git/objects`. Traslada el archivo `kotlin.txt` al área de preparación (Staging Area / Index), marcándolo como listo para ser incluido en el próximo snapshot **(SENA, 2024)**.

COMANDO 3.3 `git commit -m "Añadido datos."`

Explicación Técnica Rigurosa: Toma la instantánea de los archivos actualmente indexados en el Staging Area y genera un objeto de confirmación inalterable respaldado por una clave hash SHA-1 de 40 caracteres hexadecimales. El argumento `-m` inyecta el mensaje formal en la memoria histórica del repositorio, vinculando los metadatos del autor (`user.name` y `user.email`) y el timestamp de ejecución **(Git, 2021)**.

COMANDO 3.4 `git push github main`

Explicación Técnica Rigurosa: Ejecuta una transferencia incremental de tipo *delta*. Compara el nuevo commit local contra el estado guardado en el servidor remoto, sube únicamente los bloques de datos diferenciales y avanza el puntero de la rama remota de manera segura (Fast-Forward), actualizando la vista pública en la interfaz web de GitHub/GitLab.

5. CONCLUSIONES

- La vinculación entre el repositorio local y la plataforma remota mediante alias (`git remote add`) optimiza la administración de endpoints de red y formaliza la arquitectura distribuida del proyecto.
- La comprensión del ciclo de vida de los datos (Working Directory $\rightarrow$ Staging Area $\rightarrow$ Git Directory $\rightarrow$ Remote Server) permite manipular con precisión los estados de los archivos sin riesgo de inconsistencias.
- La utilización de herramientas CLI acelera la adopción de flujos de trabajo profesionales y constituye la base para la automatización de canalizaciones de Integración Continua (CI/CD) en la formación ADSO.

6. REFERENCIAS BIBLIOGRÁFICAS

Atlassian. (s.f.). *Qué es el control de versiones | Atlassian Git Tutorial*. Recuperado de **https://www.atlassian.com/es/git/tutorials/what-is-version-control**

Git. (2021). *Git - Acerca del control de versiones*. Documentación Oficial de Git. Recuperado de **https://git-scm.com/book/es/v2**

SENA. (2024). *Integración continua*. Componente Formativo, Grupo de Ejecución de la Formación Virtual, Servicio Nacional de Aprendizaje (SENA).